

DATA 550 Lab 2 - EDA on movie data

There are two versions of this lab, one in Python and one in R. The R lab will use `ggplot` and the Python lab will use `Altair`.

This is the Python version.

Each partner should choose a version to complete, though keep in mind that **you are required to alternate between completing the R labs and the Python labs** to get experience using both languages.

Acknowledgements

This lab was lightly adapted from the one created by Joel Ostblom for the DSCI 531 version of Data Viz I in the 2020/21 Academic year.

Submission instructions

```
rubric={mechanics:2}
```

You receive marks for submitting your lab correctly, please follow these instructions:

- Each partner should complete one file, though collaboration is permitted.
- Push your `.ipynb` files to GitHub frequently (i.e. not just in one fell swoop at the end).
- Don't forget to submit a clickable link to your GitHub repo on Canvas so we can grade it.

```
In [1]: # Run this cell to ensure that altair plots show up in the exported HTML
import altair as alt

# Save a vega-lite spec and a PNG blob for each plot in the notebook
alt.renderers.enable('mimetype')
# Handle large data sets without embedding them in the notebook
alt.data_transformers.enable('vegafusion')
alt.renderers.enable('html')
```

```
Out[1]: RendererRegistry.enable('html')
```

0. Get motivated!

I have heard that the movie market is a lucrative business, and have been looking into starting my own online streaming service together with a few friends. Before we do this, we need to understand the movie market better so that we can get a competitive edge over existing service. I have been looking all over for an aspiring data wrangler and future VIZARD, but haven't been able to track one down. This is where you come in!



Together, we must understand the current movie market, which companies produces what kind of movies, and which receive good scores. Then we can have the best movies on our platform and show only those, easy right? I mean...we will show only good movies, I can't believe no one has thought of this before! I have already identified a unique name for this service: Bestflix™.

BESTFLIX

Don't worry, we're in this together and I will be helping you with the wrangling

throughout most exercises if you take charge of the viz part, deal?. Also, I have never failed to start a company before, so you're in good hands. I can already smell the \$\$\$, so let's get started!

In this notebook, we will focus on Exploratory Data Analysis (EDA), both for single numerical columns, pairwise numerical columns, numerical columns conditioned on categorical columns and combinations of categorical columns.

1. Single numerical columns

Question 1.1

rubric={reasoning:1,accuracy:1}

Python

In the beginning of EDA, it can be good to look at some textual summaries, just to get an idea of what to plot. The data file is caled `lab2-movies.json`. Read it in and then using dataframe methods, do the following in three different cells:

1. Display the first few and last few rows of the data.
2. Display info about all columns, data types, and number of NaN values.
3. Display a summary description of the data frame's numerical columns.

```
In [1]: import json
import pandas as pd
import altair as alt
```

```
In [2]: movies = pd.read_json("data/lab2-movies.json")
```

```
In [3]: movies.head()
```

Out [3]:		id	title	runtime	budget	revenue	vote_average	vote_count
0	12		Finding Nemo	100	94000000	940335536	3.86	33887
1	13		Forrest Gump	142	55000000	677945399	4.05	91921
2	14		American Beauty	122	15000000	356296601	4.13	57879
3	16		Dancer in the Dark	140	12800000	40031879	3.84	4571
4	22		Pirates of the Caribbean: The Curse of the Bla...	143	140000000	655011224	3.81	36337

In [4]: `movies.tail()`

Out [4]:		id	title	runtime	budget	revenue	vote_average	vote_c
1003	376290		Miss Sloane	132	13000000	3500605	3.63	
1004	384682		Office Christmas Party	105	45000000	114501299	3.00	
1005	393457		Fences	139	24000000	64414761	3.50	
1006	393519		Raw	99	3500000	4100000	3.48	
1007	417644		CHiPS	100	25000000	26800152	2.68	

In [5]: `movies.info()`


```
<class 'pandas.core.frame.DataFrame'>
Index: 1008 entries, 0 to 1007
Data columns (total 9 columns):
#      Column              Non-Null Count  Dtype
---  -
0     id                    1008 non-null   int64
1     title                 1008 non-null   object
2     runtime               1008 non-null   int64
3     budget                1008 non-null   int64
4     revenue               1008 non-null   int64
5     vote_average          1008 non-null   float64
6     vote_count            1008 non-null   int64
7     genres                1008 non-null   object
8     studios               1008 non-null   object
dtypes: float64(1), int64(5), object(3)
memory usage: 78.8+ KB
```

```
In [6]: movies.describe()
```

	id	runtime	budget	revenue	vote_average
count	1008.000000	1008.000000	1.008000e+03	1.008000e+03	1008.000000
mean	49888.699405	112.183532	5.940012e+07	1.878759e+08	3.240198
std	88325.959056	21.527725	5.525947e+07	2.449621e+08	0.471919
min	12.000000	63.000000	5.000000e+00	8.500000e+01	1.610000
25%	3627.000000	96.000000	2.000000e+07	4.127994e+07	2.947500
50%	10432.500000	108.000000	4.000000e+07	1.018510e+08	3.265000
75%	44658.750000	125.000000	8.000000e+07	2.270864e+08	3.590000
max	417644.000000	202.000000	3.800000e+08	2.787965e+09	4.430000

Question 1.2

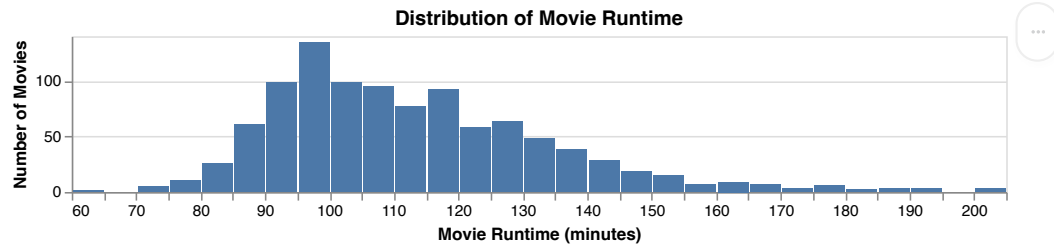
```
rubric={viz:1,accuracy:1}
```

Python

Create a single histogram of a numerical column of your choice, choosing an appropriate number of bins and set the figure height to 100.

[illegible]

Out [7]:



Question 1.3

rubric={viz:2,accuracy:2}

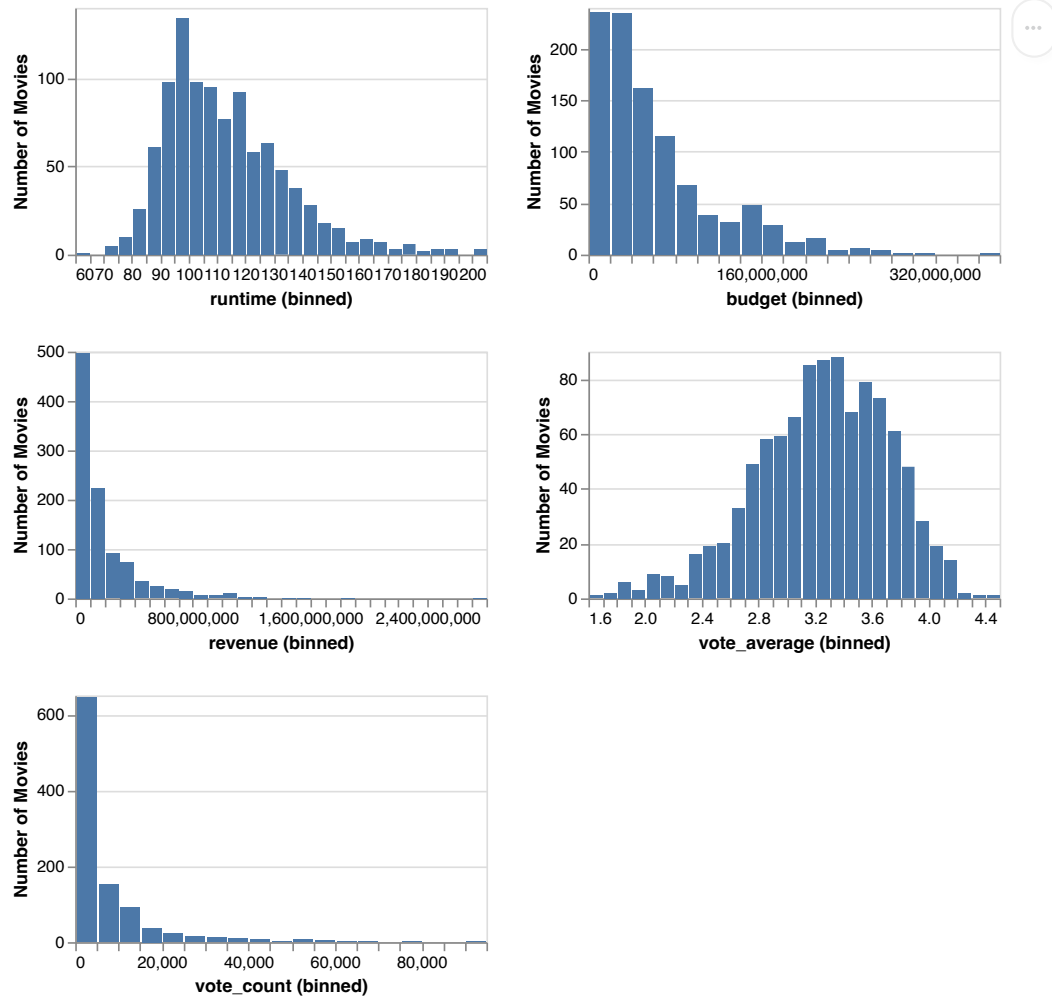
Python

- Extract the column names from all numerical columns except `id` and save them to a variable as a list (the `.select_dtypes()` method can help here, or you can do it manually).
- Copy the code from your histogram above and paste it here. Modify the code so it plots one histogram per numeric column in a plot grid with 2 columns and 3 rows. Don't use a loop, use the Altair approach to repeating charts instead (with the `repeat` keyword rather than `row` or

```
In [8]: num_cols = movies.select_dtypes(include='number').columns.drop('id').t

alt.Chart(movies).mark_bar().encode(
    alt.X(alt.repeat(), type='quantitative', bin=alt.Bin(maxbins=30)),
    y=alt.Y('count()', title='Number of Movies')
).properties(
    width=250,
    height=150
).repeat(
    repeat=num_cols,
    columns=2
)
```

Out[8]:



2. Pairwise numerical columns

Question 2.1

rubric={viz:2,accuracy:2}

Python

Next, let's look at the relationships between pairs of numerical columns. For this we will create a scatterplot matrix (SPLOM) for all numerical columns except id.

1. I recommend starting with creating a single scatter plot and gradually substituting in the repeating columns.
2. Do not include zero on either axis unless the data actually starts at zero.
3. Reduce each subplots height and width to be able to see the entire lower diagonal in view.

4. Set the opacity and size of the points directly inside `mark_point()` in order for the plots to appear less saturated.
5. Configure the axes to remove or shrink the font of the axes labels (and optional reduce the axes titles' size). Plots don't have to look perfect during EDA, the point here is to pick up trends, not necessarily to read all labels.

You can [study this example](#) if you want guidance.

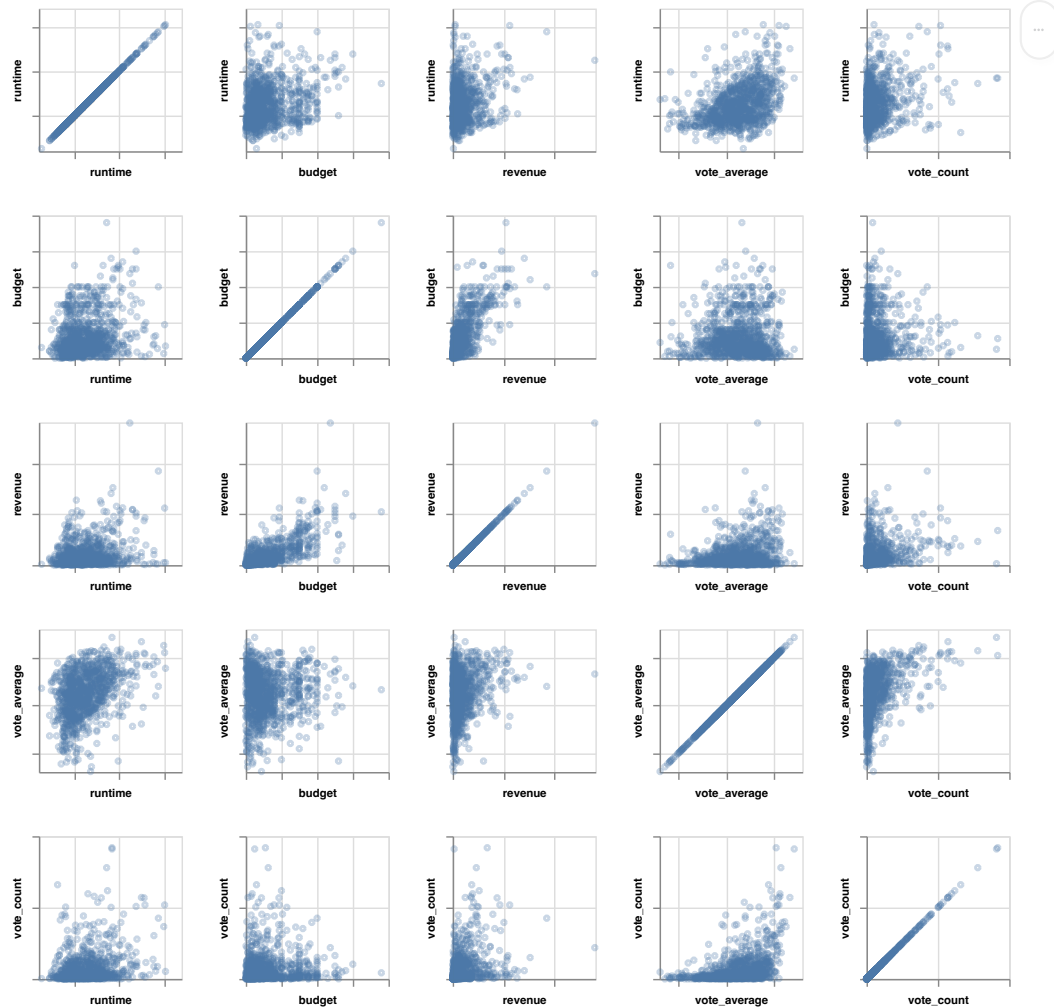
```
In [9]: # YOUR ANSWER GOES HERE

splom = alt.Chart(movies).mark_point(
    opacity=0.3,
    size=10
).encode(

    x=alt.X(alt.repeat("column"), type='quantitative', scale=alt.Scale(
    y=alt.Y(alt.repeat("row"), type='quantitative', scale=alt.Scale(ze
).properties(
    width=100,
    height=100
).repeat(
    row=num_cols,
    column=num_cols
).configure_axis(
    labelFontSize=0,
    titleFontSize=8
)

splom
```

Out [9]:



Question 2.2 (Optional)

```
rubric={viz:0.5,accuracy:0.5}
```

Python

Another way of viewing the pairwise relationships between numerical column is to assess their correlation coefficients. It can be a good idea to use spearman instead of pearson correlation in order to pick up nonlinearities, so make sure to use that below.

I have done most of the wrangling, but you need to fill in the one blank and then view the dataframe to figure out how to plot it. Use some type of point plot with a marker of your choice and make an informed decision of which encoding channels to map the different data frame columns to.

In [10]: `# YOUR ANSWER GOES HERE`

```
# corr_df = movies[numeric_cols].corr(______).stack().reset_index(na
```

Question 2.3 (Optional)

rubric={viz:0.5,accuracy:0.5}

Python

In the lecture, I mentioned that 2D histograms are a good alternative when you scatter plots become oversaturated. Copy your code from 2.1 and change it so that we can see multiple "heatmap-like" 2D histograms instead of scatter plots.

In [11]: # YOUR ANSWER GOES HERE

3. EDA of numerical columns conditioned on a categorical column (genres)

Question 3.1

rubric={viz:1,accuracy:1}

Python

Lots of interesting variation in the histograms above! I wonder if some of it can be explained by movie genre or which studio made the film... This could be valuable information for Bestify™ so let's find out!

We'll start with the movie genres. If you look at the data frame, you can see that each film has multiple genres in a list. This is not ideal and mean we must make a decision if we are counting the film once per genre or once overall. Since we have no information about which is the main genres, I suggest we go ahead and count it once per genre. In order to do this we need to replicate each row once per genre in the genres column. Luckily there is a pandas method called `explode` for exactly this. I have already done the necessary wrangling here, but please view the df so that you understand what happens and [check out pandas doc for a simpler example](#).

1. Use this new dataframe and create multiple boxplots inside a single figure by have genres on the y-axis and revenue on the x axis.
2. Boxplots can only be sorted by passing an explicit list of the sort column,

in this case the genres. I want you to sort the genres by median revenue, so that the median lines of the box plot are nicely sorted. I have given you a skeleton below where you can fill in the blanks, but you are also free to do it your own way. In the end you need a list that you can pass to `sort` inside `alt.Y()`.

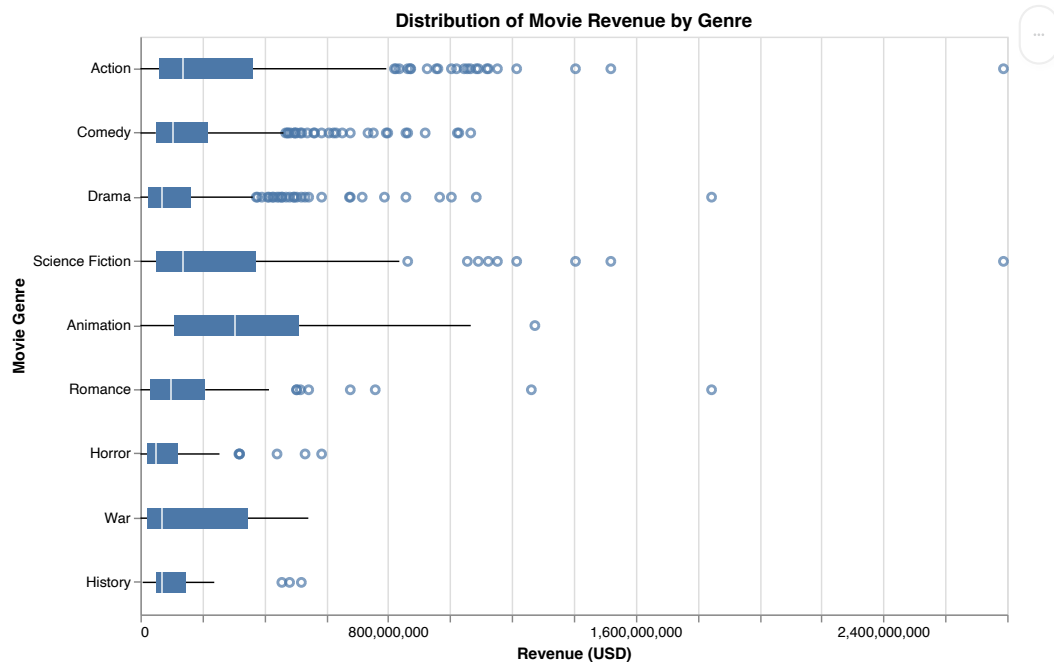
Once you have made the boxplot, you will see some interesting outliers. Remember that you can view what films these are by setting `tooltip='title'`, enjoy exploring!

```
In [13]: # Uncomment and run these lines
boom_genres = movies.explode('genres')
## Sorting by revenue because it is all about the money, money, money.
genres_by_revenue = genres_by_revenue = boom_genres.groupby('genres')[

# YOUR ANSWER GOES HERE
boxplot = alt.Chart(boom_genres).mark_boxplot().encode(
    x=alt.X('revenue:Q', title='Revenue (USD)'),
    y=alt.Y('genres:N', sort=genres_by_revenue, title='Movie Genre'),
    tooltip='title'
).properties(
    title='Distribution of Movie Revenue by Genre',
    width=600,
    height=400
)

boxplot
```

Out[13]:



Question 3.2

```
rubric={viz:1,accuracy:2,reasoning:2}
```

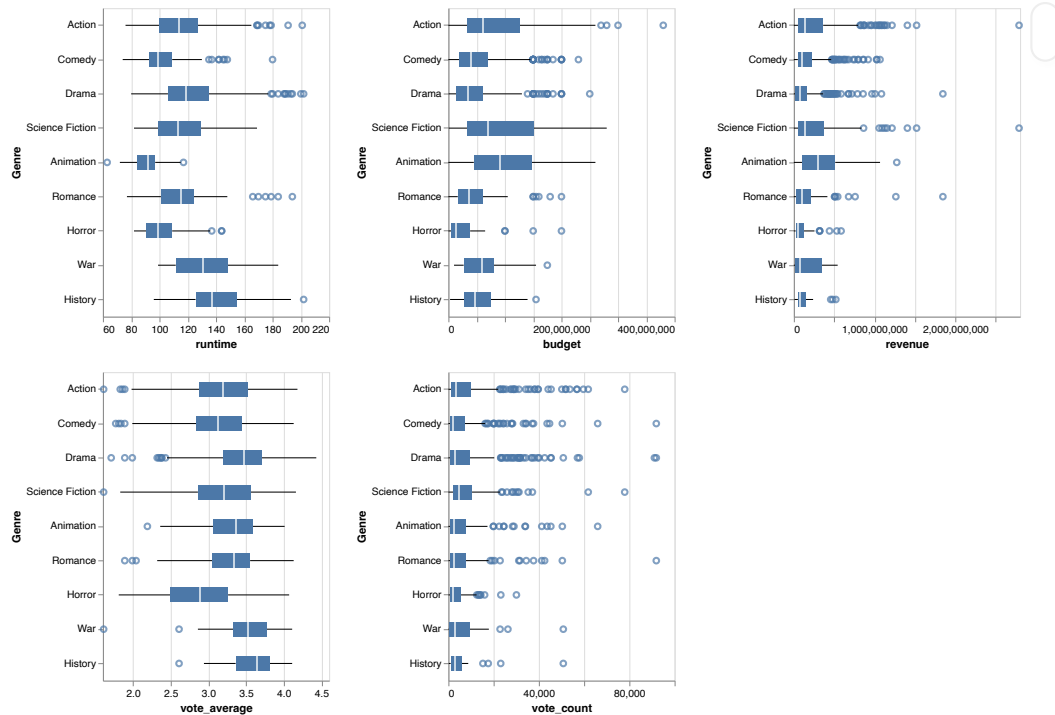
Python

1. Copy the code from your boxplot above and use the same strategy as for the histograms to repeat this boxplot for all numerical values. Make sure to set the x-axis scale to not include 0 unless necessary, so that the `runtime` boxplot uses the available space well (you don't need to sort the y-axis by different columns, it is better that categories are sorted by median revenue in all facets so that we don't have to keep track of how they move around between subplots).
2. Study the boxplots you just created and reflect over what you see. Identify a research oriented question that you would be interested in exploring and briefly motivate why you think this would be interesting (<90 words, it can be a general question, does not have to relate to Bestflix). This is an open ended question because EDA is often open ended. And even when you go in with one particular hypotheses, you must stay open to what the data tells you and be able to detect interesting relationships that you did not foresee ahead of time.

```
In [15]: # YOUR ANSWER GOES HERE
repeated_boxplots = alt.Chart(boom_genres).mark_boxplot().encode(
    x=alt.X(alt.repeat(), type='quantitative', scale=alt.Scale(zero=False)),
    y=alt.Y('genres:N', sort=genres_by_revenue, title='Genre'),
    tooltip='title'
).properties(
    width=220,
    height=300
).repeat(
    num_cols,
    columns=3
)

repeated_boxplots
```


Out[15]:



Looking at these boxplots, it's really interesting to see the disconnect between a movie's budget and how much people actually like it. Even though "big" genres like Adventure and Animation have huge revenue numbers and massive budget ranges, their audience ratings are surprisingly similar to much cheaper genres like Documentaries or Dramas.

This makes me wonder: does a movie's Return on Investment (ROI) actually correlate with its rating, or are people who watch low-budget films just naturally harder to please? It's a research question worth asking because it helps us figure out if you can actually "buy" a higher rating with high production value, or if there's a point where spending more money doesn't actually make the movie any better in the audience's eyes

Question 3.3 (Optional)

rubric={viz:0.5,accuracy:0.5}

Python

Remember that boxplots hide how many observations there are in each group, so you would need to check this manually. Before coding this answer, glance at your plot above again and reflect for yourself: which do you think is the biggest and smallest category and how many observations does there appear to be in each? Can you tell at all?

Now let's find out the answers to those questions. Make a barplot with the counts on the x-axis and the genres on the y-axis sorted after count and with the biggest bar closest to the x-axis.

In [16]: *# YOUR ANSWER GOES HERE*

Question 3.4 (Optional)

rubric={accuracy:0.5,reasoning:0.5}

Python

Let's see how the numerical columns vary with different production studios.

1. Create a new dataframe from exploding/unnesting the `studios` column in the `movies` dataframe. You can use my code from above to guide you.
2. Create a similar sorting list as before, but this time with the studies sorted by median revenue.
3. Create the same repeated boxplots for the studios. A tip when you are creating similar plots for two different sections of your notebook, is to use a different color for each so that it is easy to orient yourself (unless you are encoding color as a visual aesthetic of course, but we're not doing that here). Pick any HTML color either via its hex code or by choosing a name from [this list of named colors](#). Color all the bars and boxes in this color by setting it inside `mark_boxplot()` / `geom_boxplot()`.
4. Create the same barplot of counts as above for the studios. Color the bars in the same colors as the boxplots.
5. Reflect on the information in the boxplot and / or the barplot. Identify two interesting relationships / questions / discoveries that stand out to you and that you would want to explore further. Briefly motivate why (<90 words).

In [17]: *# YOUR ANSWER GOES HERE FOR THE REPEATED BOXPLOT*

In [18]: *# YOUR ANSWER GOES HERE FOR THE BAR PLOT OF COUNTS*

YOUR ANSWER to #5 GOES HERE

4. EDA of categorical columns

Question 4.1

rubric={viz:1,accuracy:1,reasoning:1}

Python

As the final step of our EDA, let's evaluate if studios preferentially produce some movie genres over others.

1. Explode both the `'studio'` and the `'genres'` columns via method chaining. We need both since we are counting the combinations.
2. Encode the x and y axis with the genres and studios, respectively. Map the count to color and size and plot with a mark of your choice.
3. In a new cell, switch to use `mark_rect()` and encode only color as the count.
4. Which of the plots in 2 and 3 do you think gives you the clearest understanding of how the distribution of genres within each studio?

```
In [19]: studio_genre_exploded = movies.explode('studios').explode('genres')

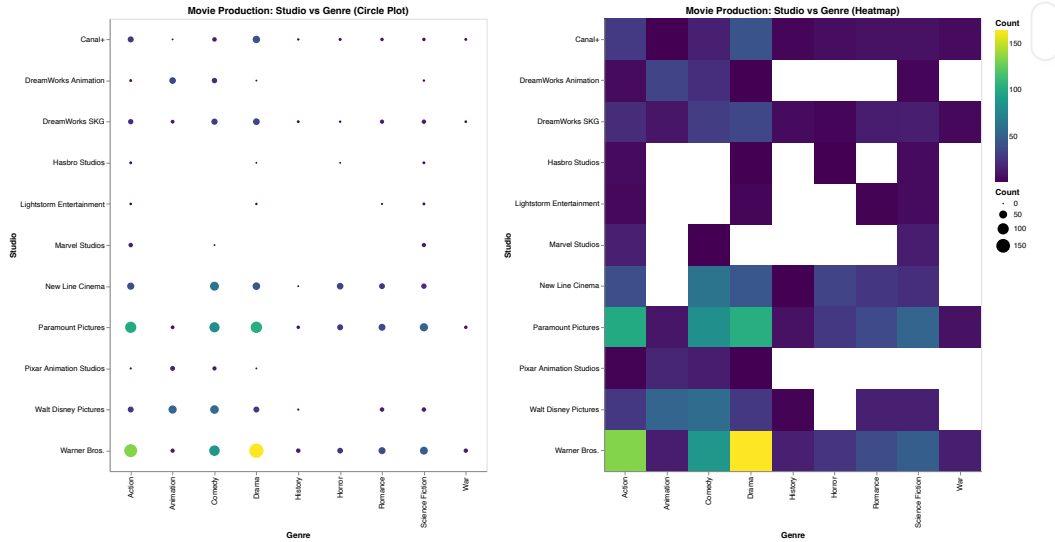
# 2. Plot with count mapped to color and size (using mark_circle)
plot_circles = alt.Chart(studio_genre_exploded).mark_circle().encode(
    x=alt.X('genres:N', title='Genre'),
    y=alt.Y('studios:N', title='Studio'),
    color=alt.Color('count()', title='Count', scale=alt.Scale(scheme='
size=alt.Size('count()', title='Count'),
    tooltip=['studios', 'genres', 'count()']
).properties(
    title='Movie Production: Studio vs Genre (Circle Plot)',
    width=500,
    height=600
)

# 3. Plot with mark_rect() and count mapped only to color (Heatmap)
plot_heatmap = alt.Chart(studio_genre_exploded).mark_rect().encode(
    x=alt.X('genres:N', title='Genre'),
    y=alt.Y('studios:N', title='Studio'),
    color=alt.Color('count()', title='Count', scale=alt.Scale(scheme='
    tooltip=['studios', 'genres', 'count()']
).properties(
    title='Movie Production: Studio vs Genre (Heatmap)',
    width=500,
    height=600
```

```
)

# Display the plots
plot_circles | plot_heatmap
```

Out[19]:



I think the heatmap (mark_rect) is much clearer. While the circle plot uses both size and color to show the count, the solid blocks in the heatmap make it much easier to identify the "signatures" of different studios. You can immediately see which genres a specific studio dominates without having to compare the diameters of floating circles.

Question 4.2 (Optional)

rubric={accuracy:1}

Python

The plots above are great for comparing absolute counts, but that means that studios with smaller production volume gets drowned out. Let's instead visualize the proportion within each studio for each genre. Calculate the proportion for each studio - genre pair, so that for each studio, the genre proportions adds up to 1. Visualize with your favorite categorical count plot.

In []: # YOUR ANSWER GOES HERE

Submission to Canvas

When you are ready to submit your assignment do the following:

1. Run all cells in your notebook to make sure there are no errors by doing
`Kernel -> Restart Kernel and Run All Cells...`
2. Commit and push all your work (including the .ipynb file) to GitHub.
3. Submit a link to your repo on Canvas (this has to be done only once per group).